

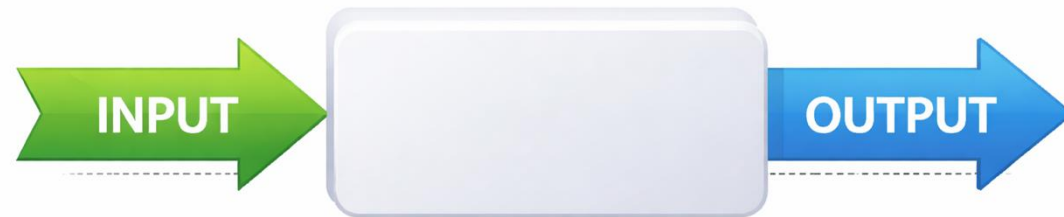
2. Modules and subworkflows

Nextflow from zero to hero

Modularity

and why it is your friend

- **Problem:** monolithic pipelines are hard to scale and maintain
- **Solution:** modularity
 - Maintainability
 - Code reuse across pipelines
 - Independent testing of components
 - Clear code organization
 - Community collaboration



Scope of the “box”

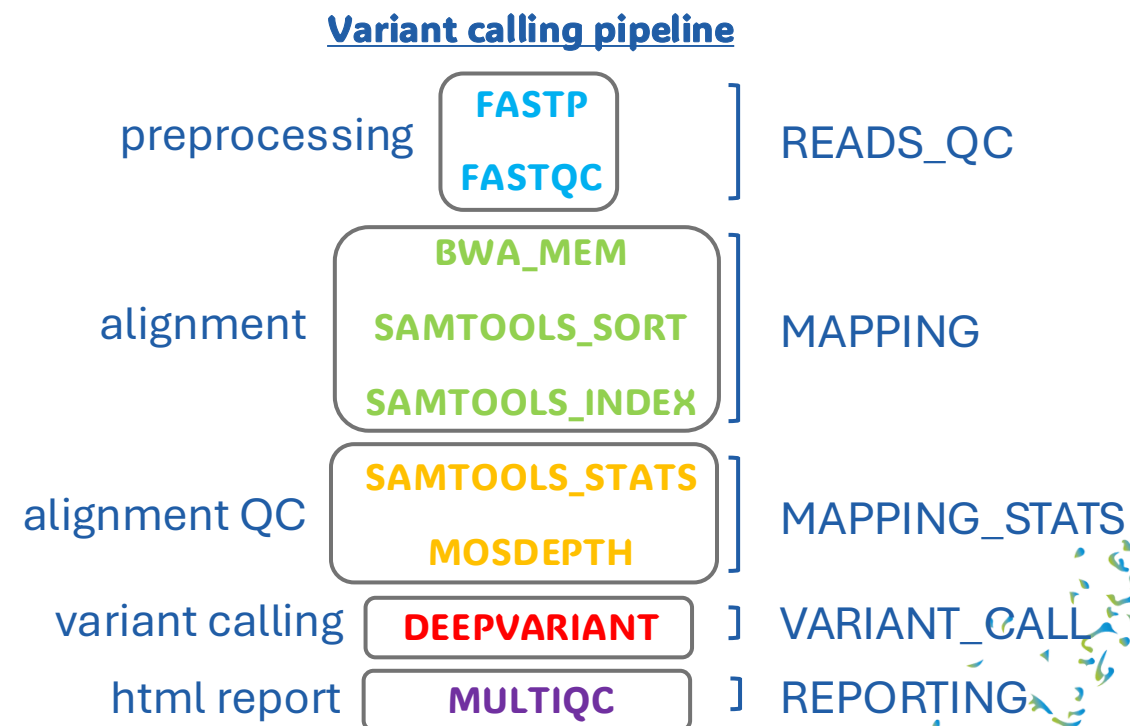
Modules and Subworkflows

– Module

Process that is stored in a **separate file** and can be imported into different workflows.

– Subworkflow

Higher-level building block that groups **multiple modules** into a **logical unit of execution**, also in a separate file.



Creating a module

- Put the process in a separate, independent file with extension *.nf* (ie. *bwa.nf*)
- Input, logic and output of the module are controlled with `input`, `script` and `output` in the process definition.
- A module can have multiple processes



modules/local/bwa/main.nf

```
process BWA_MEM {  
  
  input:  
  tuple val(sample_id), path(fastq_R1), path(fastq_R2), val(genome_id), path(genome_fasta), path(genome_indexes)  
  
  output:  
  tuple val(sample_id), path("${sample_id}.bam"), emit: bam_file  
  tuple val(sample_id), path("${sample_id}.bwa.log"), emit: bwa_log  
  tuple val("${task.process}"), val('bwa'), eval('bwa 2>&1 | tail -n+3 | head -1 | cut -d" " -f2'), topic: versions  
  tuple val("${task.process}"), val('samtools'), eval('samtools --version | head -n 1 | cut -d" " -f2'), topic: versions  
  
  script:  
  """"  
  bwa mem -t 4 \  
  -R "@RG\tID:${sample_id}\tSM:${sample_id}\tPL:Illumina" \  
  ${genome_fasta} \  
  ${fastq_R1} ${fastq_R2} \  
  2> ${sample_id}.bwa.log \  
  | samtools view --threads 4 -Sb -> ${sample_id}.bam  
  """"  
}
```

Creating a subworkflow

- **Import** the modules using `include` and the relative path
- Create a **named workflow**.
- Input, logic and output of the subworkflow are controlled with `take`, `main` and `emit` in the workflow definition.



workflows/local/mapping/main.nf

```
include { BWA_MEM } from '../modules/local/bwa'  
include { SAMTOOLS_SORT } from '../modules/local/samtools-sort'
```

Preview DAG

```
workflow MAPPING {  
  
  take:  
    qc_reads  
    genome_indexes  
  
  main:  
    bwa_input_ch = qc_reads.combine(genome_indexes)  
    BWA_MEM(bwa_input_ch)  
    bam_files_ch = BWA_MEM.out.bam_file  
  
    SAMTOOLS_SORT(bam_files_ch)  
  
  emit:  
    sorted_bam = SAMTOOLS_SORT.out.sorted_bam  
  
}
```

Creating the *main.nf*

- **Import** the subworkflows using `include` and the relative path
- Create an **UNnamed** workflow.
- As a best practice, *parameters* should be used in the **entry workflow** and passed to workflows, processes, and functions as explicit inputs.

 *main.nf*

```
include { READS_QC } from './workflows/reads-qc'
include { MAPPING } from './workflows/alignment'
include { ALIGNMENT_QC } from './workflows/alignment-qc'
include { REPORT } from './workflows/reporting'
```

Preview DAG

workflow {

```
    // Prepare reference genome indices
```

```
    genome_indexes = channel.fromPath(params.reference_genome, checkIfExists: true) ...
```

```
    // Read samplesheet with the sample ids and fastq paths
```

```
    def row_counter = 0
```

```
    input_fastq_ch = channel.fromPath(params.input_file) ...
```

```
    READS_QC(input_fastq_ch)
```

```
    MAPPING(
```

```
        READS_QC.out.qced_reads,
```

```
        genome_indexes
```

```
    )
```

```
    ALIGNMENT_QC(
```

```
        MAPPING.out.sorted_bam,
```

```
    )
```

```
    ch_multiqc_files = READS_QC.out.json_report
```

```
        .mix(ALIGNMENT_QC.out.stats_file)
```

```
        .mix(ALIGNMENT_QC.out.mosdepth_files)
```

```
        .map { _sample_id, report_file -> report_file }
```

```
        .collect()
```

```
    REPORT(ch_multiqc_files)
```

```
}
```

humantec

Same module, multiple times

- Import the module with an **alias**

```
include { cat as cat_alpha } from './some/module'
include { cat as cat_beta } from './other/module'

workflow {
    cat_alpha(some_data)
    cat_beta(other_data)
}
```

Atomicity

- One tool, one module. One functional block, one subworkflow

```
process BWA_MEM {  
  
  input:  
  tuple val(sample_id), path(fastq_R1), path(fastq_R2), val(genome_id), path(genome_fasta), path(genome_indexes)  
  
  output:  
  tuple val(sample_id), path("${sample_id}.bam"), emit: bam_file  
  tuple val(sample_id), path("${sample_id}.bwa.log"), emit: bwa_log  
  tuple val("${task.process}"), val('bwa'), eval('bwa 2>&1 | tail -n+3 | head -1 | cut -d" " -f2'), topic: versions  
  tuple val("${task.process}"), val('samtools'), eval('samtools --version | head -n 1 | cut -d" " -f2'), topic: versions  
  
  script:  
  """"  
  bwa mem -t 4 \  
  -R "@RG\tID:${sample_id}\tSM:${sample_id}\tPL:Illumina" \  
  ${genome_fasta} \  
  ${fastq_R1} ${fastq_R2} \  
  2> ${sample_id}.bwa.log \  
  | samtools view --threads 4 -Sb - > ${sample_id}.bam  
  """"  
}
```



modules/local/bwa/main.nf

Modularity

and why it is your friend

- **Problem:** monolithic pipelines are hard to scale and maintain
- **Solution:** modularity
 - Maintainability
 - Code reuse across pipelines
 - Independent testing of components
 - Clear code organization
 - Community collaboration

